

BAB IV

SHAPEGGEN: GENERATOR DATASET DAN UJI COBA GENERATOR

Bab ini menjelaskan tentang ShapeGGen dan uji coba generator dataset GraphXAI yaitu ShapeGGen yang menciptakan dataset graf dengan ground-truth yang relevan untuk penelitian.

4.1 Tentang ShapeGGen

ShapeGGen adalah komponen kunci dalam GraphXAI yang bertindak sebagai generator dataset sintetis yang dirancang khusus untuk studi explainability dalam GNN. Salah satu masalah utama dalam evaluasi explainability adalah bahwa dataset yang tersedia sering kali tidak memiliki ground-truth yang jelas untuk digunakan sebagai acuan dalam menilai kualitas penjelasan model. ShapeGGen mengatasi masalah ini dengan menghasilkan dataset graf yang telah diproses dan dilengkapi dengan ground-truth yang dapat digunakan untuk benchmarking metode explainability.

Kemampuan utama ShapeGGen adalah fleksibilitasnya dalam menghasilkan berbagai jenis graf dengan beragam karakteristik, seperti graf homofilik dan heterofilik, serta graf dengan ground-truth kecil maupun besar. Selain itu, ShapeGGen juga memungkinkan pembuatan dataset dengan tingkat fairness yang dapat dikontrol, sehingga dapat digunakan dalam studi explainability yang berfokus pada fairness dan interpretabilitas model GNN dalam berbagai skenario aplikasi. Dengan kemampuannya ini, ShapeGGen menjadi alat yang sangat penting dalam penelitian di bidang explainability GNN, karena memungkinkan eksperimen yang lebih terkontrol dan dapat direproduksi.

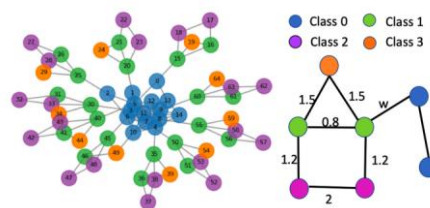
4.2 Jenis Dataset yang Dihasilkan oleh ShapeGGen

Dalam ShapeGGen, dataset dapat dikategorikan berdasarkan sifat hubungan antar-node, ukuran ground-truth explanation, fairness dalam struktur graf, serta kompleksitas informasi yang tersedia dalam fitur node. Berikut adalah beberapa jenis dataset yang dapat dihasilkan oleh ShapeGGen dan bagaimana dataset ini digunakan dalam penelitian explainability.

4.2.1 SG-Base (Dataset Dasar untuk Explainability)

SG-Base adalah versi dasar dari dataset yang dihasilkan oleh ShapeGGen. Dataset ini bersifat homofilik, berukuran besar, dan mengandung motif berbentuk rumah (house-shaped motifs) sebagai bagian dari ground-truth explanations. Motif ini dipilih karena merupakan salah satu bentuk sintetik paling awal yang digunakan dalam penelitian explainability GNN.

SG-Base memiliki lebih dari 10.000 node, sehingga menyediakan jumlah contoh yang cukup besar untuk evaluasi metode explainability secara statistik. Dengan sifatnya yang homofilik, dataset ini mencerminkan karakteristik jaringan sosial, di mana node dengan atribut yang mirip cenderung terhubung satu sama lain.



Gambar 4.1
House Shaped Motif Graph¹

Gambar di atas menunjukkan house-shaped motif, yaitu motif berbentuk rumah yang sering digunakan dalam dataset ShapeGGen untuk penelitian explainability dalam Graph Neural Networks (GNNs). Struktur ini terdiri dari lima node yang saling terhubung, membentuk pola menyerupai rumah. Dua node di

¹ Xiaoxiao Li, Jaoao Saude. Explain Graph Neural Networks to Understand Weighted Graph Features in Node Classification, hlm 68.

bagian atas merepresentasikan atap rumah, sementara tiga node lainnya di bagian bawah membentuk dinding dan fondasi rumah. Motif ini digunakan sebagai ground-truth explanation dalam eksperimen explainability karena merupakan salah satu bentuk sintetik paling awal yang digunakan dalam evaluasi model GNN.

Angka yang tertera pada setiap edge (garis antar-node) menunjukkan bobot hubungan antara node dalam graf. Bobot ini memiliki beberapa fungsi tergantung pada konteks penggunaannya. Dalam graf berbobot (weighted graph), bobot menunjukkan seberapa kuat koneksi antar-node, di mana nilai yang lebih tinggi mengindikasikan hubungan yang lebih signifikan dibandingkan dengan nilai yang lebih rendah. Misalnya, bobot 1.5 menunjukkan hubungan yang lebih kuat dibanding bobot 0.8. Selain itu, dalam sistem jaringan sosial atau sistem rekomendasi, bobot dapat menunjukkan seberapa sering dua node berinteraksi atau seberapa relevan hubungan antar-node. Dalam Graph Neural Networks (GNNs), bobot edge memainkan peran penting dalam proses propagasi informasi (message passing). Setiap node dalam GNN memperoleh representasi berdasarkan informasi dari node tetangganya, di mana bobot edge menentukan seberapa besar pengaruh informasi yang diteruskan antar-node. Jika sebuah edge memiliki bobot tinggi, informasi dari satu node ke node lain akan memiliki dampak yang lebih besar dibandingkan edge dengan bobot rendah. Misalnya, dalam jaringan sosial, bobot edge dapat merepresentasikan seberapa kuat hubungan antar-individu, sedangkan dalam sistem rekomendasi, bobot edge bisa menunjukkan seberapa sering dua item atau pengguna berinteraksi. Oleh karena itu, dalam banyak model GNN seperti Graph Convolutional Networks (GCN) dan Graph Attention Networks (GAT), bobot edge digunakan untuk memberikan perhatian yang berbeda pada setiap hubungan dalam graf.

Namun, meskipun bobot edge sangat penting dalam banyak model GNN, ShapeGGen secara default menghasilkan graf yang tidak berbobot (unweighted graph). Ini berarti setiap edge dalam graf memiliki pengaruh yang sama tanpa adanya bobot yang membedakan hubungan antar-node. Keputusan ini diambil karena banyak metode explainability dalam GNN bekerja dengan asumsi bahwa semua koneksi antar-node memiliki pengaruh yang setara. Dengan menggunakan

graf tanpa bobot, penelitian explainability dapat lebih fokus pada bagaimana model memahami struktur graf itu sendiri tanpa dipengaruhi oleh perbedaan bobot edge. Selain itu, banyak dataset nyata yang digunakan dalam penelitian GNN juga bersifat unweighted, terutama graf yang hanya mencatat hubungan tanpa memberi nilai bobot tertentu, contohnya graf pertemanan di Facebook.

Meskipun ShapeGGen tidak menyertakan bobot edge secara bawaan, pengguna tetap dapat menambahkan bobot edge secara manual setelah graf dibuat jika eksperimen memerlukannya. Salah satu cara untuk melakukannya adalah dengan menambahkan atribut bobot ke dalam adjacency matrix atau ke dalam edge attributes dalam format PyTorch Geometric.

Ada beberapa situasi di mana graf berbobot lebih relevan dalam penelitian explainability. Misalnya, dalam graf transportasi, bobot edge dapat merepresentasikan jarak atau waktu perjalanan, sehingga model perlu memahami hubungan antar-node dengan mempertimbangkan bobot yang berbeda. Selain itu, dalam model seperti GAT (Graph Attention Networks), bobot edge sering kali dihitung secara dinamis berdasarkan tingkat kepentingan antar-node. Oleh karena itu, jika penelitian ingin mengevaluasi metode explainability dalam konteks graf berbobot, maka menambahkan bobot setelah dataset ShapeGGen dibuat bisa menjadi solusi yang efektif. Berikut parameter untuk menghasilkan dataset SG-Base:

$$G = \text{ShapeGGen}(S = \text{'house'}, N_s = 1200, p = 0.006, n_s = 11, K = 2, n_f = 11, n_i = 4, s_f = 0.6, c_f = 2, \varphi = 0.5, \eta = 1, L = 3)$$

SG-Base sering digunakan dalam penelitian awal untuk mengevaluasi sejauh mana metode explainability dapat menangkap pola yang sudah diketahui dalam graf. Sebagai contoh, jika suatu metode tidak mampu mengidentifikasi subgraf yang relevan dalam SG-Base, maka metode tersebut kemungkinan akan mengalami kesulitan dalam dataset yang lebih kompleks.

4.2.2 SG-Heterophilic (Dataset untuk Evaluasi dalam Graf Heterofilik)

SG-Heterophilic adalah dataset yang dirancang untuk menguji metode explainability dalam graf heterofilik, yaitu graf di mana node yang terhubung memiliki karakteristik yang sangat berbeda. Hal ini berbeda dengan graf homofilik, di mana node yang terhubung biasanya memiliki kemiripan dalam fitur atau labelnya.

Sebagian besar metode explainability dalam GNN diuji pada graf homofilik, karena model GNN bekerja lebih baik dalam graf dengan homofili tinggi. Namun, dalam dunia nyata, banyak dataset yang bersifat heterofilik, di mana node yang terhubung cenderung memiliki karakteristik yang sangat berbeda. Berikut parameter untuk menghasilkan dataset SG-Heterophilic:

$$G = \text{ShapeGGen}(S = \text{'house'}, N_s = 1200, p = 0.006, n_s = 11, K = 2, n_f = 11, n_i = 4, s_f = 0.6, c_f = 2, \varphi = 0.5, \eta = -1, L = 3)$$

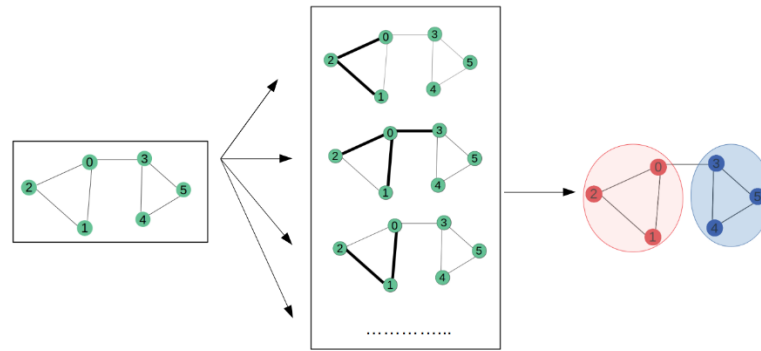
SG-Heterophilic sangat berguna dalam penelitian explainability karena banyak metode explainability yang lebih mudah bekerja dalam graf homofilik. Dengan menggunakan dataset ini, pengguna dapat mengevaluasi apakah suatu metode explainability tetap efektif dalam lingkungan yang tidak beraturan.

4.2.3 SG-SmallEx & SG-LargeEx (Dataset Berdasarkan Ukuran Ground-Truth Explanation)

Salah satu aspek penting dalam evaluasi explainability adalah seberapa besar ground-truth explanations dalam dataset. Dalam banyak kasus, dataset yang tersedia hanya memiliki ground-truth explanation dalam ukuran kecil, yang mungkin tidak cukup untuk menilai metode explainability secara menyeluruh. ShapeGGen dapat menghasilkan dataset dengan ukuran ground-truth explanation yang berbeda, yaitu SG-SmallEx (penjelasan berukuran kecil) dan SG-LargeEx (penjelasan berukuran besar).

- SG-SmallEx menggunakan motif segitiga sebagai ground-truth explanation, yang ukurannya lebih kecil.

- SG-LargeEx menggunakan motif rumah, yang lebih besar dan lebih kompleks, sehingga menguji apakah metode explainability dapat tetap bekerja dengan baik ketika ukuran explanation bertambah.



Gambar 4.2
Triangle Shaped Motif Graph²

Triangle motif graph adalah struktur sederhana dengan tiga node yang saling terhubung dalam satu siklus tertutup, sering digunakan dalam analisis keterhubungan dalam jaringan sosial dan deteksi komunitas. Di sisi lain, house motif graph adalah struktur yang lebih kompleks, terdiri dari lima node dan enam edge yang membentuk pola rumah, yang lebih sering digunakan dalam penelitian explainability untuk mengidentifikasi pola yang lebih besar dalam dataset GNNs. House motif lebih menantang untuk dipahami oleh model dibandingkan triangle motif, sehingga sering digunakan dalam eksperimen GraphXAI dan ShapeGGen untuk mengevaluasi metode explainability dalam GNN. Berikut parameter untuk menghasilkan dataset SG-SmallEx:

$$G = \text{ShapeGGen}(S = \text{'triangle'}, Ns = 1200, p = 0.006, ns = 12, K = 2, nf = 11, ni = 4, sf = 0.5, cf = 2, \varphi = 0.5, \eta = 1, L = 3)$$

Parameter SG-LargeEx tidak disebutkan secara eksplisit, hanya dijelaskan secara konsep sebagai dataset dengan ground-truth explanation yang lebih besar menggunakan house-shaped motifs.

² Graph Convolutional Network (GCN). Hoonst (<https://hoonst.github.io/2020/12/26/Graph-Convolutional-Network/> diakses pada tanggal 15 Februari 2025)

Dataset ini sering digunakan untuk mengetahui apakah metode explainability tetap akurat ketika ukuran explanation bertambah besar. Jika suatu metode bekerja baik dalam SG-SmallEx tetapi gagal dalam SG-LargeEx, maka metode tersebut mungkin mengalami overfitting terhadap pola kecil dan tidak mampu menangani penjelasan yang lebih luas.

4.2.4 SG-Unfair (Dataset untuk Evaluasi Bias dalam Explainability)

Salah satu tantangan utama dalam evaluasi explainability adalah fairness, yaitu sejauh mana model GNN dapat memberikan penjelasan yang adil terhadap semua kelompok dalam dataset. Bias dalam model dapat menyebabkan ketidakadilan, misalnya jika model lebih cenderung mengandalkan fitur tertentu yang tidak relevan untuk membuat prediksi.

SG-Unfair adalah dataset yang dibuat untuk mengontrol fairness properties dalam GNN. Dengan ShapeGGen, kita dapat menciptakan dataset yang memiliki tingkat fairness yang dapat dikendalikan, termasuk weakly-unfair dan strongly-unfair graphs.

- SG-Base digunakan sebagai contoh weakly-unfair graphs, di mana terdapat sedikit bias dalam dataset.
- SG-Unfair digunakan untuk menciptakan strongly-unfair graphs, yang dapat digunakan untuk mengevaluasi sejauh mana metode explainability dapat mempertahankan fairness dalam prediksi GNN.

Berikut parameter untuk menghasilkan dataset SG-Unfair:

$$G = \text{ShapeGGen}(S = \text{'house'}, N_s = 1200, p = 0.006, ns = 11, K = 2, nf = 11, ni = 4, sf = 0.6, cf = 2, \varphi = 0.75, \eta = 1, L = 3)$$

SG-Unfair digunakan dalam penelitian fairness dalam explainability. Misalnya, jika suatu metode explainability selalu memberikan fokus tinggi pada fitur yang seharusnya netral dalam keputusan model, maka metode tersebut mungkin tidak sepenuhnya adil dan perlu disesuaikan agar lebih objektif.

4.2.5 SG-MoreInform & SG-LessInform (Dataset Berdasarkan Kualitas Informasi Fitur Node)

Metode explainability sering kali diuji dalam kondisi di mana fitur yang tersedia dalam node sudah cukup kaya dan berisi informasi yang relevan untuk klasifikasi. Namun, dalam kondisi dunia nyata, tidak semua dataset memiliki fitur yang cukup informatif, dan beberapa fitur mungkin tidak berkontribusi terhadap label node.

SG-MoreInform dan SG-LessInform dibuat untuk menguji bagaimana jumlah informasi dalam fitur node mempengaruhi hasil explainability.

- SG-MoreInform memiliki persentase fitur informatif yang lebih tinggi (8 dari 11 fitur node berkaitan dengan label).
- SG-LessInform memiliki persentase fitur informatif yang lebih rendah (4 dari 21 fitur berkaitan dengan label), sehingga lebih sulit bagi metode explainability untuk menemukan pola yang benar.
- SG-Base memiliki proporsi fitur informatif di antara keduanya (4 dari 11 fitur).

Berikut parameter untuk menghasilkan dataset SG-MoreInform:

$$G = \text{ShapeGGen}(S = 'house', Ns = 1200, p = 0.006, ns = 11, K = 2, nf = 11, ni = 8, sf = 0.6, cf = 2, \varphi = 0.5, \eta = 1, L = 3)$$

Dan berikut ini adalah parameter untuk dataset SG-LessInform:

$$G = \text{ShapeGGen}(S = 'house', Ns = 1200, p = 0.006, ns = 11, K = 2, nf = 21, ni = 4, sf = 0.6, cf = 2, \varphi = 0.5, \eta = 1, L = 3)$$

Dataset ini sangat berguna dalam penelitian explainability untuk melihat seberapa sensitif metode explainability terhadap jumlah informasi yang tersedia dalam fitur node. Jika suatu metode hanya bekerja dengan baik dalam SG-MoreInform tetapi gagal dalam SG-LessInform, maka metode tersebut mungkin terlalu bergantung pada fitur eksplisit dan tidak cukup fleksibel dalam menangkap pola yang lebih kompleks.

4.3 Parameter dalam ShapeGGen

Berikut adalah parameter yang dapat diatur untuk generate data sesuai keinginan, pada awalnya graph didefinisikan sebagai

$$G = \text{ShapeGGen}(S, Ns, p, ns, K, nf, ni, sf, cf, \varphi, \eta, L)$$

Penjelasan setiap parameter dapat dilihat di bawah ini:

1. **S** (Motif)

S adalah motif atau subgraf yang ditanamkan dalam graf yang dihasilkan. Motif ini bisa berupa pola tertentu seperti triangle motif (segitiga) atau house motif (rumah). Motif ini berfungsi sebagai ground-truth explanation untuk mengevaluasi performa metode explainability dalam GNN.

Contoh: Jika $S = \text{'triangle'}$, maka subgraf berbentuk segitiga akan digunakan dalam graf yang dihasilkan.

2. **Ns** (Jumlah Subgraf)

Ns adalah jumlah subgraf yang digunakan dalam proses pembentukan graf awal. Semakin besar nilai **Ns**, semakin banyak motif yang akan ditanamkan dalam graf.

Contoh: Jika $Ns = 100$, maka graf yang dihasilkan akan memiliki 100 subgraf dengan motif tertentu.

3. **p** (Probabilitas Koneksi Antar-Subgraf)

p adalah probabilitas koneksi antar-subgraf, yang memengaruhi panjang jalur terpendek rata-rata dalam graf. Nilai **p** yang lebih kecil untuk **Ns** yang besar lebih disarankan untuk menghindari graf yang terlalu padat, yang dapat menyebabkan performa GNN menurun.

Contoh: Jika $p = 0.01$, maka koneksi antar-subgraf akan lebih jarang dibandingkan dengan $p = 0.1$.

4. **ns** (Ukuran Subgraf)

ns adalah ukuran yang diharapkan dari setiap subgraf dalam proses pembuatan graf ShapeGGen. Nilai yang besar akan menghasilkan graf dengan distribusi derajat yang lebih panjang.

Contoh: Jika $ns = 10$, maka setiap subgraf dalam graf yang dihasilkan akan memiliki sekitar 10 node.

5. **K** (Jumlah Kelas)

K menunjukkan jumlah kelas node dalam dataset, yang memengaruhi tugas klasifikasi dalam model GNN. Semakin banyak kelas (K lebih besar), semakin sulit tugas klasifikasinya, karena model harus membedakan lebih banyak kelompok node.

Contoh: Jika $K = 3$, dataset memiliki tiga jenis label node, sedangkan jika $K = 5$, maka model harus mengenali lebih banyak variasi dalam label node.

6. **nf** (Dimensi Fitur Node)

nf adalah jumlah total fitur yang dimiliki oleh setiap node dalam dataset. Semakin besar nilai **nf** , semakin banyak informasi yang tersedia bagi model GNN dalam melakukan klasifikasi.

Contoh: Jika $nf = 16$, maka setiap node dalam graf memiliki 16 fitur unik, yang dapat digunakan sebagai input dalam model GNN.

7. **ni** (Jumlah Fitur Informatif)

ni adalah jumlah fitur node yang benar-benar berguna untuk klasifikasi, sementara fitur lainnya hanya berupa noise atau tidak relevan. Jika **ni** lebih kecil dibanding **nf** , tugas klasifikasi menjadi lebih sulit karena hanya sebagian kecil fitur yang informatif.

Contoh:

Jika **$nf = 10$** dan **$ni = 3$** , maka hanya 3 dari 10 fitur yang berguna untuk klasifikasi.

Jika **$ni = 8$** , lebih banyak fitur yang berguna, sehingga tugas klasifikasi menjadi lebih mudah.

8. **sf** (Faktor Pemisahan Kelas)

sf menentukan seberapa jelas fitur node dapat membedakan kelas yang berbeda. Jika **sf** rendah, fitur antar-kelas sulit dibedakan, membuat klasifikasi lebih sulit. Jika **sf** tinggi, kelas lebih mudah dipisahkan berdasarkan fitur node.

Contoh:

Jika $sf = 0.2$, maka fitur antar-kelas hampir tidak bisa dibedakan.

Jika $sf = 0.9$, kelas lebih mudah dibedakan.

9. **cf** (Jumlah Cluster per Kelas)

cf menentukan berapa banyak cluster yang ada dalam setiap kelas node.

Contoh:

Jika $cf = 1$, semua node dalam satu kelas membentuk satu kelompok besar.

Jika $cf = 3$, setiap kelas akan terbagi ke dalam 3 cluster lebih kecil, meningkatkan kesulitan klasifikasi.

10. ϕ (Protected Feature Noise Factor)

ϕ menentukan seberapa besar fitur yang dilindungi (protected feature) memiliki korelasi dengan label. Parameter ini digunakan untuk mengontrol fairness dalam dataset.

Contoh:

Jika $\phi = 0$, fitur yang dilindungi memiliki korelasi penuh dengan label.

Jika $\phi = 0.5$, fitur yang dilindungi memiliki korelasi sebagian.

Jika $\phi = 1$, fitur yang dilindungi memiliki korelasi negatif dengan label.

11. η (Homophily Coefficient)

η menentukan seberapa homofilik atau heterofilik graf yang dihasilkan.

Contoh:

Jika $\eta > 0$, node dengan label yang sama cenderung terhubung satu sama lain (graf homofilik).

Jika $\eta < 0$, node dengan label berbeda lebih sering terhubung (graf heterofilik).

12. L (Jumlah Layer dalam Model GNN)

L adalah jumlah lapisan dalam model GNN yang menentukan seberapa jauh informasi menyebar dalam graf.

Contoh:

Jika $L = 1$, model hanya bisa mengakses tetangga langsung dari setiap node.

Jika $L = 3$, model bisa menangkap informasi dari tetangga hingga jarak 3 hop

4.4 Analisis Source Code ShapeGGen

ShapeGGen adalah generator dataset graf yang dirancang untuk penelitian explainability dalam Graph Neural Networks (GNNs). Untuk memahami bagaimana ShapeGGen bekerja, diperlukan analisa struktur kode sumbernya,

termasuk bagaimana parameter dikonfigurasi, bagaimana graf dibangun, serta bagaimana ground-truth explanations dihasilkan.

Struktur kelas utama dalam ShapeGGen meliputi:

- Konstruktor (`__init__`) – Menginisialisasi parameter dan konfigurasi dataset.
- Fungsi `generate_shape_graph()` – Membangun graf berdasarkan motif dan parameter yang diberikan.
- Fungsi `explanation_generator()` – Menghasilkan ground-truth explanations untuk explainability GNN.
- Fungsi `visualize()` – Menampilkan graf hasil generasi dalam bentuk visual.

Setiap bagian dari kode memiliki peran penting dalam memastikan dataset yang dihasilkan memiliki karakteristik yang sesuai dengan kebutuhan eksperimen explainability dan akan dijelaskan pada subbab selanjutnya.

4.1 Konstruktor ShapeGGen

Segmen program Konstruktor ShapeGGen dapat dilihat pada Lampiran A.1. Konstruktor `__init__` bertanggung jawab untuk menginisialisasi parameter utama yang akan digunakan dalam pembuatan dataset graf. Parameter ini mencakup jumlah kelas, jenis motif, ukuran dataset, dan apakah ground-truth explanation akan dihasilkan. Pada baris dua, model layers menentukan jumlah lapisan dalam model GNN yang akan digunakan untuk mempelajari dataset (value defaultnya 3). Di baris empat, motif yang akan digunakan dalam graf, misalnya house motif atau triangle motif. Pada baris enam, seed digunakan untuk reproducibility, memastikan dataset yang dihasilkan tetap sama setiap kali dijalankan. Selanjutnya di baris delapan, terdapat boolean yang menentukan apakah ground-truth explanations akan dibuat. Dan kwargs adalah parameter tambahan yang dapat dikonfigurasi lebih lanjut. Pada baris 13, konstruktor ini memanggil `super().__init__()` yang menginisialisasi kelas induk dan menetapkan jumlah hops dalam model GNN berdasarkan parameter `model_layers`. Parameter `name = 'ShapeGGen'` digunakan untuk menyimpan nama model dataset dan `num_hops = model_layers` menentukan seberapa jauh informasi dalam graf menyebar, sesuai jumlah lapisan model.

Di baris 16-20, digunakan untuk inisialisasi variabel dasar. `Self.in_shape` untuk menyimpan informasi tentang bentuk motif yang digunakan. `Self.graph = None` untuk inisialisasi objek graf kosong yang nantinya akan diisi dengan node dan edge. `Self.model_layers` untuk menyimpan jumlah lapisan model. `Self.make_explanations` untuk menyimpan informasi apakah ground-truth explanation akan dibuat.

Pada baris 22-44, untuk memastikan parameter dapat dikustomisasi dengan fleksibel, konstruktor membaca nilai dari `kwargs` dan menetapkan nilai default jika parameter tidak diberikan. Parameter `variant` menentukan metode yang digunakan untuk membangun graf dalam `ShapeGGen`. Jika `variant = 0`, maka metode `BBG_old` digunakan, sedangkan jika `variant = 1`, graf dibentuk menggunakan metode `Preferential Attachment (BBG_PA)`. Selain itu, `num_subgraphs = 10` menunjukkan jumlah subgraf yang akan ditanamkan dalam dataset, yang masing-masing memiliki ukuran tertentu sesuai dengan parameter `subgraph_size = 13`, yang menentukan jumlah node dalam setiap subgraf. Selanjutnya, `prob_connection = 1` mengontrol seberapa besar kemungkinan dua subgraf akan terhubung satu sama lain, yang dapat mempengaruhi struktur jaringan dan tingkat keterhubungan antar-node. Jenis graf dasar yang digunakan dalam dataset ditentukan oleh `base_graph = 'ba'`, yang berarti graf ini dibangun menggunakan model Barabási-Albert (BA), yaitu model graf skala-bebas dengan distribusi derajat yang mengikuti pola power-law. Untuk memastikan motif dalam dataset terbentuk dengan benar, `ShapeGGen` memiliki opsi verifikasi motif, yang dikendalikan oleh `verify = True`. Jika verifikasi diaktifkan, sistem akan melakukan pengecekan apakah motif yang diinginkan berhasil dimasukkan ke dalam graf. Jika proses ini gagal, `ShapeGGen` akan mencoba membangun graf ulang hingga `max_tries_verification = 5`, yang berarti sistem akan melakukan maksimal lima kali percobaan sebelum akhirnya memberikan error jika graf yang valid tidak dapat dibuat.

Pada baris 46-65 berfungsi untuk inisialisasi parameter fitur. Parameter `n_informative = 4` menentukan jumlah fitur node yang benar-benar berkontribusi terhadap klasifikasi, sementara fitur lainnya dapat dianggap sebagai noise atau tidak relevan dalam menentukan label node. Selain itu, `class_sep = 1.0` berfungsi sebagai

faktor pemisahan kelas, yang menentukan seberapa jelas perbedaan antara kelas berdasarkan fitur node. Semakin tinggi nilai `class_sep`, semakin mudah model untuk membedakan kelas yang berbeda. Dataset dalam ShapeGGen juga memiliki total fitur yang ditentukan oleh `n_features = 10`, di mana hanya sebagian dari fitur tersebut yang benar-benar informatif dalam proses klasifikasi. Setiap kelas dalam dataset dapat terbagi menjadi beberapa cluster, yang dikendalikan oleh `n_clusters_per_class = 2`, yang berarti setiap kelas akan terdiri dari dua kelompok atau cluster dengan distribusi yang berbeda. Terakhir, `homophily_coef = None` menunjukkan bahwa koefisien homofili tidak ditentukan secara eksplisit, sehingga dataset dapat bersifat homofilik (node dengan label yang sama cenderung terhubung) atau heterofilik (node dengan label berbeda lebih sering terhubung) tergantung pada nilai yang diberikan untuk parameter ini.

Pada baris 67-79 digunakan untuk inisialisasi fitur sensitive. Parameter `add_sensitive_feature = True` menentukan apakah fitur sensitif akan ditambahkan ke dalam dataset. Fitur sensitif ini dapat digunakan dalam penelitian fairness dalam explainability, terutama untuk mengevaluasi apakah model GNN memperlakukan kelompok tertentu secara berbeda berdasarkan atribut yang tidak seharusnya memengaruhi prediksi. Jika `attribute_sensitive_feature = True`, maka fitur sensitif akan dikontrol berdasarkan label node, yang berarti ada hubungan langsung antara atribut sensitif dengan kelas yang diberikan, memungkinkan studi tentang bias dalam klasifikasi. Selain itu, `sens_attribution_noise = 0.25` mengontrol tingkat noise dalam atribusi fitur sensitif, di mana semakin tinggi nilai noise, semakin kecil pengaruh fitur sensitif terhadap label, sehingga mengurangi kemungkinan model bergantung pada fitur tersebut dalam prediksi.

Pada baris 84-135 berguna untuk menentukan bentuk motif yang akan dimasukkan ke dalam graf dan membangun struktur graf sesuai dengan parameter yang ditentukan. Jika parameter shape yang diberikan berupa objek `nx.Graph`, maka motif tersebut akan langsung digunakan dalam pembentukan graf tanpa perlu melalui proses pencocokan. Namun, jika shape diberikan dalam bentuk string, maka nilai tersebut akan dibandingkan dengan daftar motif yang telah didefinisikan sebelumnya. Jika motif yang dipilih adalah `house`, maka motif berbentuk rumah

akan digunakan sebagai bagian dari struktur graf yang dihasilkan. Sebaliknya, jika shape tidak sesuai dengan motif yang sudah ditentukan, maka nilai default akan tetap None. Graf dalam ShapeGGen dibangun menggunakan metode BBG_old atau BBG_PA, yang dipilih berdasarkan nilai variant yang diberikan sebagai parameter. Jika variant = 0, maka graf akan dibuat menggunakan BBG_old, sedangkan jika variant = 1, metode Preferential Attachment (BBG_PA) akan digunakan untuk membentuk graf dengan pola koneksi yang lebih realistis berdasarkan tingkat keterhubungan antar-node.

Pada baris 137-150 setelah graf terbentuk, sistem akan melakukan verifikasi motif menggunakan fungsi verify_motifs() untuk memastikan bahwa struktur yang dihasilkan sesuai dengan motif yang diinginkan. Jika verifikasi ini gagal setelah beberapa kali percobaan, kode akan memunculkan error dan menyarankan pengguna untuk mencoba menggunakan parameter yang berbeda atau meningkatkan jumlah maksimum percobaan verifikasi agar graf yang valid dapat terbentuk.

Pada baris 183-201, setelah graf berhasil dibangun dalam ShapeGGen, langkah berikutnya adalah membagi dataset ke dalam tiga subset utama, yaitu training (pelatihan), testing (pengujian), dan validation (validasi). Pembagian ini bertujuan untuk melatih model GNN, menguji performanya, serta mengevaluasi apakah model dapat melakukan generalisasi dengan baik terhadap data baru.

Segmen Program 4.1 Contoh Implementasi Konstruktor ShapeGGen

```
01: # Membuat dataset dengan house motif dan 10 subgraf
02: dataset = ShapeGGen(shape='house', num_subgraphs=10,
03:                     prob_connection=0.05)
```

Kode di atas adalah contoh konstruktor ShapeGGen yang akan menghasilkan graf dengan motif house dan 10 subgraf, di mana kemungkinan koneksi antar-subgraf hanya 5%. Hal ini memastikan bahwa graf tidak terlalu padat.

4.2 Fungsi untuk Menghasilkan Bentuk Graf

Berikut ini adalah penjelasan fungsi untuk membuat bentuk graph yang sudah diinisialisasikan parameternya pada konstruktor ShapeGGen. Segmen program `generate_shape_graph` dapat dilihat pada Lampiran A.2.

Pada baris satu sampai tujuh, fungsi ini akan menghasilkan label node berdasarkan struktur graf. Label-label tersebut kemudian dikonversi menjadi tensor PyTorch, yang memungkinkan kompatibilitas dengan framework deep learning. Salinan eksplisit dari tensor ini disimpan untuk mencegah kemungkinan bug akibat perubahan tensor secara tidak disengaja selama proses pelatihan model.

Setelah label ditetapkan, fungsi melanjutkan dengan menghasilkan fitur node menggunakan Gaussian Level Generator (baris 9-22). Proses ini menghasilkan fitur berbasis Gaussian yang digunakan untuk membangun dataset dengan variabel yang dikontrol, termasuk jumlah total fitur, jumlah fitur informatif yang benar-benar memiliki pengaruh dalam klasifikasi, serta faktor pemisahan kelas untuk memastikan ada perbedaan yang cukup antara kelas yang berbeda. Fitur-fitur ini dikumpulkan dalam bentuk tensor, yang menyusun fitur setiap node menjadi representasi yang dapat digunakan dalam model pada baris 24-26).

Pada baris 28-72, bagian ini akan mengatur fitur sensitif berdasarkan label node, di mana nilai probabilitas perubahan ditentukan oleh noise. Dalam kasus ini, fitur sensitif akan ditentukan sebagai inversi label node. Namun, jika fitur sensitif tidak dikontrol oleh label, sistem akan menghasilkan nilai acak antara 0 dan 1 untuk setiap node. Fitur ini kemudian ditambahkan ke dalam, di mana fitur sensitif disisipkan sebagai dimensi tambahan dalam representasi fitur node dan sudah di acak terlebih dahulu agar tidak ada bias urutan.

Setelah fitur node ditetapkan, pada baris 73-77 fungsi membentuk struktur hubungan antar-node dengan mengubah daftar edge dari graf ke dalam format tensor PyTorch. Proses ini dilakukan menggunakan fungsi `to_undirected()` yang memastikan bahwa hubungan antar-node dalam graf bersifat tidak berarah. Jika parameter homofili diatur (baris 79-107), fungsi akan mengoptimalkan nilai fitur untuk memastikan bahwa graf memiliki tingkat homofili yang sesuai dengan yang diinginkan. Optimalisasi ini dilakukan dengan `optimize_homophily()`, yang

menyesuaikan fitur berdasarkan konektivitas node dan label yang diberikan, serta menjalankan proses ini selama sejumlah iterasi tertentu.

Pada baris 115-136 merupakan langkah terakhir dalam fungsi ini yaitu menyusun semua informasi yang telah dikumpulkan ke dalam format PyTorch Geometric. Fungsi ini membuat objek Data() yang menyimpan fitur node (x), label (y), serta indeks hubungan antar-node (edge_index). Jika ground-truth explanation diaktifkan, maka akan dihasilkan penjelasan berbasis motif untuk setiap node. Namun, jika opsi explainability tidak diaktifkan, nilai self.explanations akan tetap kosong.

Segmen Program 4.2 Contoh Implementasi Fungsi generate_shape_graph() pada ShapeGGen

```
01: # Membuat dataset dengan ground-truth explanation
02: dataset = ShapeGGen(shape='house', num_subgraphs=15,
03:                     make_explanations=True)
04:
05: # Mengakses dataset yang telah di-generate
06: print("Jumlah Node: ", dataset.graph.x.shape[0]) # Jumlah
07: Node: 191
08: print("Jumlah Edge: ", dataset.graph.edge_index.shape[1])
09: #Jumlah Edge: 622
```

Fungsi generate_shape_graph() dipanggil dalam konstruktor untuk memastikan bahwa setiap objek ShapeGGen langsung memiliki dataset yang lengkap dan siap digunakan. Sehingga pengguna tidak perlu lagi memanggil fungsi tambahan setelah membuat objek dan proses pembuatan dataset lebih otomatis, efisien, dan minim kesalahan.

4.3 Fungsi untuk Menghasilkan Ground Truth

Fungsi explanation_generator() dalam ShapeGGen digunakan untuk menghasilkan ground-truth explanation berdasarkan hubungan antar-node dalam graf. Fungsi ini menentukan kepentingan node (node importance) dan kepentingan edge (edge importance) berdasarkan motif dalam jaringan. Tujuan utamanya adalah mengidentifikasi node dan edge mana yang paling relevan dalam memahami struktur motif dalam dataset.

Pada baris satu sampai enam, fungsi ini mencari node dalam radius satu-hop dari node pusat (`node_idx`) menggunakan `khop` subgraph. Hanya node yang memiliki motif tertentu yang akan diperhitungkan, sementara node dengan nilai `shape nol` akan diabaikan. Hasilnya disimpan dalam bentuk set, memastikan bahwa tidak ada duplikasi dalam pencatatan motif yang ditemukan.

Setelah menentukan node dalam radius satu-hop, fungsi memperluas pencariannya ke radius lebih besar sesuai dengan jumlah lapisan model. Node-node yang ditemukan dibandingkan dengan daftar motif yang telah dikumpulkan sebelumnya. Jika suatu node memiliki bentuk motif yang sama dengan node dalam radius satu-hop, node tersebut dianggap penting dan disimpan dalam dictionary `node_imp_map`, yang berisi pasangan node dan tingkat kepentingannya dalam jaringan (baris 8-16).

Segmen Program 4.3 Fungsi untuk Menghasilkan Penjelasan pada ShapeGGen

```

01: def explanation_generator(self, node_idx):
02:     # Menentukan node yang termasuk dalam num_hops
03:     tertentu di sekitar node target
04:     original_in_num_hop = set([self.G.nodes[n]['shape']
05:     for n in khop_subgraph_nx(node_idx, 1, self.G) if
06:     self.G.nodes[n]['shape'] != 0])
07:
08:     # Menentukan node mana saja dalam jangkauan
09:     model_layers hops dari node target
10:     khop_nodes = khop_subgraph_nx(node_idx,
11:     self.model_layers, self.G)
12:
13:     # Membuat mapping untuk menentukan apakah suatu node
14:     termasuk dalam motif yang dianalisis
15:     node_imp_map = {i:(self.G.nodes[i]['shape'] in
16:     original_in_num_hop) for i in khop_nodes}
17:
18:     # Mendapatkan informasi subgraf dari node target
19:     dalam jangkauan num_hops
20:     khop_info = k_hop_subgraph(
21:         node_idx,
22:         num_hops = self.model_layers,
23:         edge_index =
24:         to_undirected(self.graph.edge_index)
25:     )
26:
27:     node_imp = torch.tensor([node_imp_map[i.item()] for
28:     i in khop_info[0]], dtype=torch.double)
29:
30:     # Menentukan pentingnya edge dalam motif

```

Segmen Program 4.3 (Lanjutan)

```

31:         in_motif = khop_info[0][node_imp.bool()] # Get nodes
32:         in the motif
33:         edge_imp = torch.zeros(khop_info[1].shape[1],
34:                                dtype=torch.double)
35:         for i in range(khop_info[1].shape[1]):
36:             # Jika kedua node dalam edge termasuk dalam
37:             motif, maka edge tersebut dianggap penting
38:             if (khop_info[1][0,i] in in_motif) and
39:             (khop_info[1][1,i] in in_motif):
40:                 edge_imp[i] = 1
41:                 continue
42:             # Memastikan memberi highlight
43:             pada edge yang menghubungkan node target
44:             one_edge_in_motif = ((khop_info[1][0,i] in
45:                                   in_motif) or (khop_info[1][1,i] in in_motif))
46:             node_idx_in_motif = (node_idx in in_motif)
47:             one_end_of_edge_is_nidx = ((khop_info[1][0,i] ==
48:                                         node_idx) or (khop_info[1][1,i] == node_idx))
49:
50:             if (one_edge_in_motif and
51:                 one_end_of_edge_is_nidx) and (not
52:                 node_idx_in_motif):
53:                 edge_imp[i] = 1
54:
55:         # Membuat objek Explanation yang berisi informasi
56:         pentingnya fitur, node, dan edge
57:         exp = Explanation(
58:             feature_imp=self.feature_imp_true,
59:             node_imp = node_imp,
60:             edge_imp = edge_imp,
61:             node_idx = node_idx
62:         )
63:
64:         # Menetapkan subgraf yang mengelilingi node yang
65:         Dianalisis dan mengembalikan hasil explanation
66:         exp.set_enclosing_subgraph(khop_info)
67:         return [exp]

```

Selanjutnya, fungsi ini membangun subgraf dalam radius multi-hop. Fungsi ini mengambil semua node dan edge yang terhubung dengan `node_idx` dalam radius hop dan memastikan bahwa setiap edge dianggap tidak berarah (undirected). Dengan subgraf ini, fungsi kemudian menentukan kepentingan setiap node dalam subgraf dengan mengambil nilai kepentingan dari `node_imp_map` dan mengonversinya ke dalam tensor PyTorch, sehingga dapat digunakan dalam pemrosesan lebih lanjut (baris 18-28).

Setelah menentukan node yang relevan, fungsi ini mulai mengevaluasi kepentingan setiap edge dalam subgraf. Edge dianggap penting jika kedua ujungnya

Segmen Program 4.4 (Lanjutan)

```

31:         0., 0., 0., 0., 0., 0., 0., 0., 0.,
32:         0., 0., 0., 0., 0., 0., 0., 0., 0.,
33:         0., 0., 0., 0., 0., 0.],
34:         dtype=torch.float64)

```

Hasil penjelasan yang diperoleh dari `explanation_generator()` menunjukkan tiga aspek utama dalam analisis model GNN, yaitu fitur penting, node penting, dan edge penting dalam explanation. Dalam analisis fitur penting, tensor yang dihasilkan menunjukkan bahwa dari 11 fitur yang ada dalam dataset, hanya 4 fitur utama yang memiliki skor kepentingan tinggi (1), yaitu fitur pada indeks ke-5, 6, 9, dan 10. Hal ini sesuai dengan konfigurasi awal dataset yang mengatur `n_informative=4`, yang berarti memang hanya 4 fitur yang benar-benar berkontribusi dalam klasifikasi. Dengan demikian, hasil ini dapat dikatakan valid, karena model secara efektif mengenali fitur yang relevan dan mengabaikan fitur yang tidak penting.

Dalam analisis node penting, hasil tensor menunjukkan bahwa node 0 hingga 4 memiliki skor kepentingan tinggi (1), yang mengindikasikan bahwa node-node ini memiliki peran besar dalam keputusan model. Selain itu, node 18 hingga 21 juga terdeteksi sebagai node yang penting, kemungkinan karena mereka merupakan bagian dari motif yang dipelajari. Sebaliknya, sebagian besar node lainnya bernilai 0, yang berarti tidak terlalu berkontribusi dalam keputusan model. Hal ini cukup masuk akal, karena tidak semua node dalam graf memiliki pengaruh yang sama terhadap hasil klasifikasi. Model seharusnya hanya menyoroti node yang benar-benar relevan dengan motif atau pola yang sedang dipelajari, dan hasil yang diperoleh menunjukkan bahwa model telah menangkap pola tersebut dengan baik.

Pada analisis edge penting, hasil tensor menunjukkan bahwa edge-edge pada indeks awal memiliki skor kepentingan tinggi (1), yang berarti mereka menghubungkan node-node penting dalam graf. Mayoritas edge lainnya memiliki nilai 0, yang menunjukkan bahwa mereka tidak memiliki kontribusi signifikan dalam klasifikasi model. Ini sesuai dengan harapan, karena dalam motif yang digunakan (seperti house motif atau triangle motif), hanya beberapa edge yang benar-benar memiliki hubungan penting dalam pembentukan struktur utama.

Dengan demikian, hasil ini juga tampak valid, karena hanya edge yang relevan yang mendapatkan skor kepentingan tinggi, sementara koneksi lain yang tidak berkontribusi pada klasifikasi tetap diabaikan.

4.4 Fungsi untuk Visualisasi Dataset Sintetis

Fungsi `visualize()` dalam `ShapeGGen` digunakan untuk menampilkan graf yang telah dihasilkan dengan memberikan warna dan label pada node sesuai dengan bentuk motif atau nilai labelnya. Fungsi ini memungkinkan pengguna untuk melihat bagaimana node dan edge diatur dalam dataset serta apakah motif yang telah ditanam dalam graf dapat diidentifikasi dengan jelas.

Segmen Program 4.5 Fungsi Visualisasi pada `ShapeGGen`

```

01: def visualize(self, shape_label = False, ax = None,
02: show = False):
03:     # Menggunakan sumbu (axis) yang diberikan, atau
04:     membuat sumbu baru jika tidak disediakan
05:     ax = ax if ax is not None else plt.gca()
06:     # Mendapatkan daftar node dalam graf beserta
07:     atributnya
08:     Gitems = list(self.G.nodes.items())
09:     # Membuat mapping dari node ke indeksny dalam daftar
10:     node
11:     node_map = {Gitems[i][0]:i for i in
12: range(self.G.number_of_nodes())}
13:
14:     # Menentukan label node yang akan digunakan untuk
15:     pewarnaan visualisasi
16:     if shape_label:
17:         y = [int(self.G.nodes[i]['shape'] > 0) for i in
18: range(self.num_nodes)]
19:     else:
20:         ylist = self.graph.y.tolist()
21:         y = [ylist[node_map[i]] for i in self.G.nodes]
22:
23:     # Membuat dictionary untuk memberi label numerik ke
24:     setiap node dalam graf
25:     node_weights = {i:node_map[i] for i in self.G.nodes}
26:
27:     # Menentukan tata letak (layout) node dalam graf
28:     menggunakan algoritma spring layout, Seed diberikan
29:     agar hasil visualisasi selalu konsisten
30:     pos = nx.spring_layout(self.G, seed = 1234)
31:     # Menggambar graf dengan warna node sesuai dengan
32:     nilai label yang telah ditentukan
33:     nx.draw(self.G, pos, node_color = y, labels =
34: node_weights, ax=ax)
35:

```

Segmen Program 4.5 (Lanjutan)

```
36: # Jika parameter show=True, maka tampilkan hasil
37:     visualisasi graf
38:     if show:
39:         plt.show()
```

Pada baris 1-25, berfungsi untuk mengatur parameter hingga mapping node ke indeksinya. Jika parameter `shape_label` diatur ke `True`, maka node akan diberi warna berdasarkan apakah mereka merupakan bagian dari motif atau tidak, sedangkan jika `shape_label` bernilai `False`, warna node akan ditentukan berdasarkan label yang telah diberikan dalam dataset. Sebelum menggambar graf, sistem terlebih dahulu menentukan axes matplotlib yang akan digunakan. Jika tidak diberikan oleh pengguna, maka akan digunakan axes default dengan `plt.gca()`. Kemudian, ID node dengan indeks numeriknya dimapping menggunakan `node_map`.

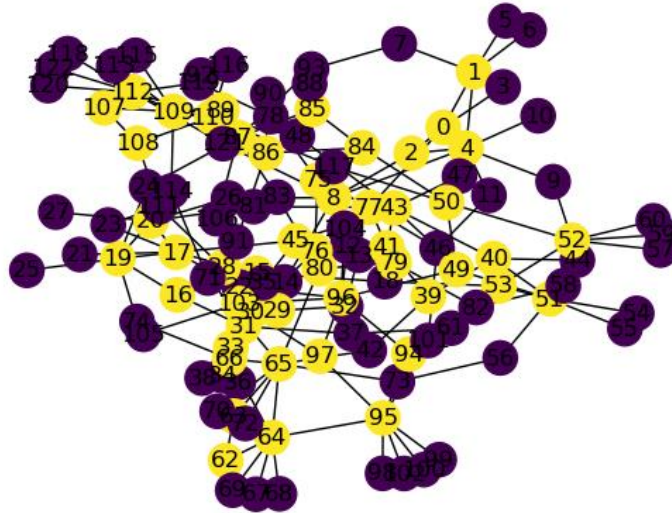
Untuk menentukan posisi setiap node dalam graf, fungsi ini menggunakan algoritma Spring Layout dari NetworkX. Dan juga penggunaan seed dengan nilai tetap untuk memastikan bahwa posisi node dalam visualisasi selalu konsisten, sehingga tampilan tidak berubah setiap kali kode dijalankan. Setelah posisi node ditetapkan, sistem menggambar graf menggunakan `nx.draw()`, dengan warna node yang sudah ditentukan sebelumnya, serta menampilkan label numerik untuk setiap node agar lebih mudah dipahami. Jika parameter `show` diatur ke `True`, maka graf akan langsung ditampilkan (baris 27-39).

Segmen Program 4.6 Contoh Implementasi Fungsi Visualisasi pada ShapeGGen

```
01: dataset = ShapeGGen(shape='house', num_subgraphs=10,
02:                     make_explanations=True)
03: dataset.visualize(shape_label=True, show=True)
```

Hasil implementasi fungsi `visualize()` pada `ShapeGGen` menghasilkan representasi graf yang menunjukkan bagaimana subgraph dengan motif house terbentuk dalam dataset. Dengan menggunakan parameter `shape_label=True`, label yang ditampilkan dalam visualisasi berdasarkan struktur motif yang digunakan dalam dataset, sehingga memudahkan analisis bagaimana motif-motif ini tersebar

dalam graf. Selain itu, dengan `show=True`, hasil visualisasi langsung ditampilkan, memungkinkan pengguna untuk memahami struktur graf secara lebih mudah.



Gambar 4.3
Visualisasi pada hasil dataset ShapeGGen

4.5 Pengujian Motif dalam Dataset ShapeGGen

Dalam bagian ini, uji coba dilakukan untuk mengevaluasi keberadaan motif house-shaped motif dan triangle motif dalam dataset yang dihasilkan oleh ShapeGGen. Pengujian ini bertujuan untuk memastikan bahwa motif yang ditanamkan dalam graf dapat dikenali dan divisualisasikan dengan baik. Dua dataset yang digunakan dalam pengujian ini adalah SG-Base untuk house-shaped motif dan SG-SmallEx untuk triangle motif.

4.5.1 House Motif Based

Dataset SG-Base dipilih untuk uji coba ini karena secara eksplisit menggunakan motif house-shaped sebagai bagian dari struktur ground-truth. Dataset ini dihasilkan dengan parameter sebagai berikut:

$$G = \text{ShapeGGen}(S = \text{'house'}, N_s = 1200, p = 0.006, ns = 11, K = 2, nf = 11, ni = 4, sf = 0.6, cf = 2, \varphi = 0.5, \eta = 1, L = 3)$$

Berikut adalah langkah-langkah yang dilakukan untuk uji coba ini. Yang pertama yaitu menghasilkan dataset SG-Base dengan ShapeGGen (Segmen

Program 4.7 baris 1-9), dan menghasilkan visualisasi graphnya yang bisa dilihat pada gambar 4.4. Penjelasan kode selanjutnya akan dijelaskan pada bagian setelah visualisasi hasil generate dataset menggunakan kode di bawah ini.

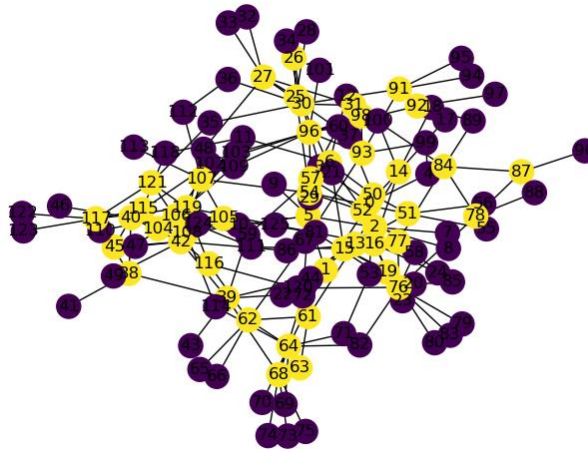
Segmen Program 4.7 Mendeteksi dan Memvisualisasikan House Motif Based pada dataset graph ShapeGGen

```

01:  from graphxai.datasets import ShapeGGen
02:  import matplotlib.pyplot as plt
03:  # Membuat dataset dengan motif house-shaped - SG-Base
04:  dataset = ShapeGGen(S='house', Ns=1200, p=0.006, ns=11, K=2,
05:                      nf=11, ni=4, sf=0.6, cf=2, φ=0.5, η=1, L=3)
06:
07:  fig, ax = plt.subplots(figsize=(8, 6))
08:  dataset.visualize(shape_label=True, ax=ax, show=True)
09:
10:  import networkx as nx
11:  from networkx.algorithms.isomorphism import GraphMatcher
12:
13:  # Definisi house-shaped motif dan mencari motif house
14:  pertama
15:  house_motif = nx.Graph()
16:  house_motif.add_edges_from([(0,1), (1,2), (2,3), (3,0),
17:                             (1,4), (2,4)]) # Struktur motif rumah
18:
19:  def find_house_motif_fast(G):
20:      matcher = GraphMatcher(G, house_motif)
21:      motifs = list(matcher.subgraph_isomorphisms_iter())
22:
23:      if len(motifs) == 0:
24:          print("House-shaped motif tidak ditemukan dalam
25:                graf.")
26:          return
27:
28:      # Ambil satu motif pertama yang ditemukan
29:      selected_motif_nodes = list(motifs[0].keys())
30:      subG = G.subgraph(selected_motif_nodes)
31:
32:      # Visualisasi motif di dalam graf
33:      plt.figure(figsize=(5, 5))
34:      pos = nx.spring_layout(subG, seed=42) # Posisi node
35:      untuk visualisasi
36:      nx.draw(subG, pos, with_labels=True,
37:              node_color='lightblue', edge_color='black',
38:              node_size=500, font_size=10)
39:      plt.title("Visualisasi House-Shaped Motif")
40:      plt.show()
41:
42:  # Konversi ShapeGGen ke NetworkX
43:  G_nx = nx.Graph()
44:  edge_list = dataset.graph.edge_index.numpy().T # Konversi
45:  edge_index dari tensor ke numpy
46:  G_nx.add_edges_from(edge_list)
47:  find_house_motif_fast(G_nx)

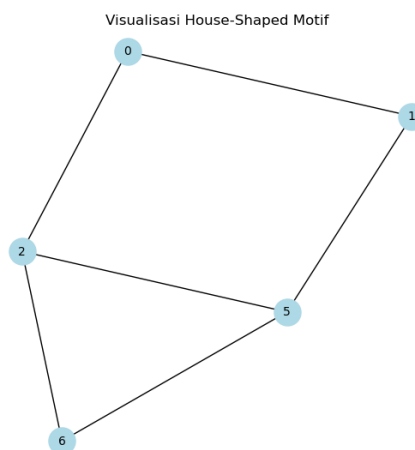
```

Hasil Visualisasi dari generate menggunakan dataset generator ShapeGGen adalah sebagai berikut:



Gambar 4.4
Hasil Generate Dataset SG-Base

Langkah selanjutnya yaitu pada baris 11-50 akan dilakukan konversi dataset ke NetworkX untuk mempermudah pencarian mencari house-shaped motif menggunakan Graph Pattern Matching (GraphMatcher), memvisualisasikan satu house motif pertama yang ditemukan dalam graf, dapat dilihat pada Gambar 4.5.



Gambar 4.5
House Shaped Motif pertama yang ditemukan

Hasil ini menunjukkan bahwa ShapeGGen memang berhasil menghasilkan motif house-shaped sesuai dengan paramter inputnya. Dengan metode pencocokan motif ini, pengguna dapat lebih memahami bagaimana motif-motif ini terdistribusi dalam graf.

4.5.2 Triangle Motif Based

Dataset SG-SmallEx dipilih untuk uji coba ini karena secara eksplisit menggunakan motif triangle-shaped sebagai bagian dari struktur ground-truth. Dataset ini dihasilkan dengan parameter sebagai berikut:

$$G = \text{ShapeGGen}(S = 'triangle', Ns = 1200, p = 0.006, ns = 12, K = 2, nf = 11, ni = 4, sf = 0.5, cf = 2, \phi = 0.5, \eta = 1, L = 3)$$

Berikut adalah langkah-langkah yang dilakukan untuk uji coba ini. Yang pertama yaitu menghasilkan dataset SG-Base dengan ShapeGGen (Segmen Program 4.8 baris 1-8), dan menghasilkan visualisasi graphnya yang bisa dilihat pada gambar 4.6.

Segmen Program 4.8 Mendeteksi dan Memvisualisasikan Triangle Motif Based pada dataset graph ShapeGGen

```
01: from graphxai.datasets import ShapeGGen
02: import matplotlib.pyplot as plt
03: # Membuat dataset dengan motif house-shaped - SG-SmallEx
04: dataset = ShapeGGen(S='triangle', Ns=1200, p=0.006, ns=12,
05:                     K=2, nf=11, ni=4, sf=0.5, cf=2, phi=0.5, eta=1, L=3)
06: # Visualisasi dataset
07: fig, ax = plt.subplots(figsize=(8, 6))
08: dataset.visualize(shape_label=True, ax=ax, show=True)
09:
10: import networkx as nx
11:
12: # Fungsi mencari dan menampilkan satu triangle motif
13: def find_and_visualize_triangle_motif(G):
14:     for node in G.nodes():
15:         neighbors = list(G.neighbors(node))
16:         for i in range(len(neighbors)):
17:             for j in range(i + 1, len(neighbors)):
18:                 if G.has_edge(neighbors[i], neighbors[j]):
19:                     # Jika ada hubungan antar dua tetangga,
20:                     # maka terbentuk triangle motif
21:                     triangle_nodes = [node, neighbors[i],
22:                                     neighbors[j]]
23:                     subG = G.subgraph(triangle_nodes)
24:                     # Visualisasi motif triangle
25:                     plt.figure(figsize=(4, 4))
```

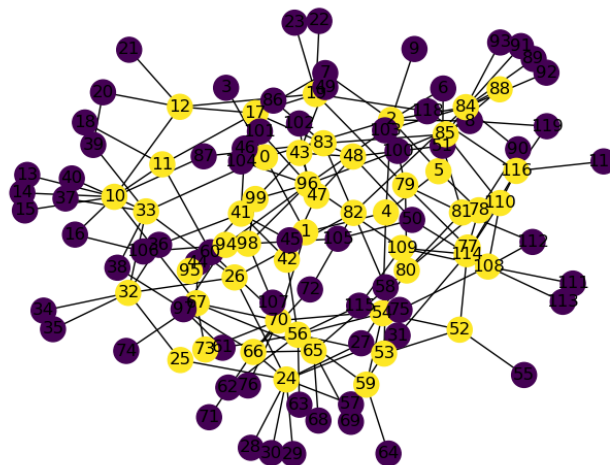
Segmen Program 4.8 (Lanjutan)

```

26:         pos = nx.spring_layout(subG, seed=42) #
27:         Posisi node untuk visualisasi
28:         nx.draw(subG, pos, with_labels=True,
29:         node_color='lightcoral',
30:         edge_color='black', node_size=500,
31:         font_size=10)
32:         plt.title("Visualisasi Triangle Motif")
33:         plt.show()
34:
35:         print(f"Triangle motif ditemukan dengan
36:         node: {triangle_nodes}")
37:         return triangle_nodes # Hanya
38:         menampilkan satu triangle motif pertama
39:         yang ditemukan
40:
41:     print("Triangle motif tidak ditemukan dalam graf.")
42:     return None
43:
44: # Konversi ShapeGGen ke NetworkX
45: G_nx = nx.Graph()
46: edge_list = dataset.graph.edge_index.numpy().T # Konversi
47: edge_index dari tensor ke numpy
48: G_nx.add_edges_from(edge_list)
49:
50: # Jalankan pencarian dan visualisasi triangle motif
51: find_and_visualize_triangle_motif(G_nx)

```

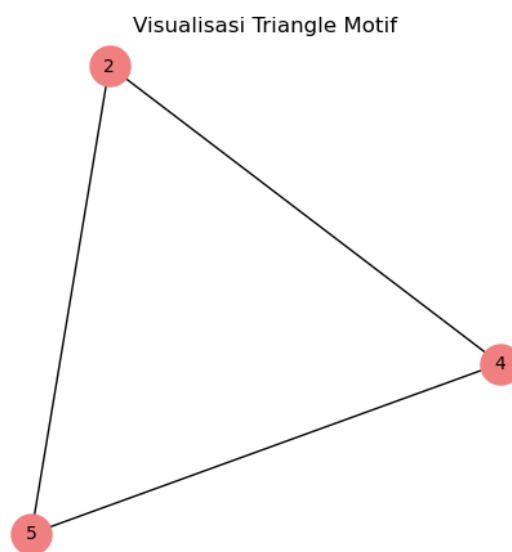
Hasil Visualisasi dari generate menggunakan dataset generator ShapeGGen adalah sebagai berikut:



Gambar 4.6
Hasil Generate Dataset SG-SmallEx

Langkah selanjutnya yaitu mengonversi dataset ke NetworkX untuk mempermudah pencarian motif, mencari triangle-shaped motif menggunakan fungsi bawaan NetworkX dan memvisualisasikan satu triangle motif pertama yang ditemukan dalam graf (baris 12-49).

Setelah memanggil fungsi `find_and_visualize_triangle_motif()`, akan menghasilkan gambar motif seperti di bawah ini berdasarkan dataset yang sudah digenerate sebelumnya (baris 52).



Gambar 4.7
Triangle Shaped Motif pertama yang ditemukan

Dengan demikian, dataset yang dihasilkan oleh ShapeGGen dapat digunakan untuk menguji metode explainability dalam GraphXAI dengan memastikan motif yang relevan telah tertanam dalam dataset.

4.6 Evaluasi Pengaruh Parameter Koefisien Homofili

ShapeGGen terhadap struktur Graph

Proses pengujian ini dapat dilihat pada segmen program 4.9 yang bertujuan untuk mengevaluasi pengaruh parameter homophily coefficient (η) pada dataset graf yang dibuat menggunakan ShapeGGen. Homophily coefficient menentukan apakah node dengan label yang sama lebih cenderung terhubung (homofilik, $\eta = 1$)

atau justru lebih sering terhubung dengan node berlabel berbeda (heterofilik, $\eta = -1$). Untuk mengetahui teori tersebut benar, dibutuhkan membuat dataset graf homofili dan heterofili menggunakan ShapeGGen. Pada uji coba ini, akan menggunakan dataset ShapeGGen homofili dan heterofili.

Pada baris 1-51, dibuat fungsi `visualize_graph()` yang memvisualisasikan graf dengan warna node berdasarkan labelnya. Setiap node diberi warna sesuai label menggunakan skala warna coolwarm, dan tata letak node dibuat tetap konsisten dengan spring layout agar visualisasi tidak berubah-ubah setiap kali dijalankan. Kemudian, fungsi `compute_homophily_ratio()` digunakan untuk menghitung rasio homofili, yaitu perbandingan jumlah edge yang menghubungkan node dengan label yang sama terhadap total edge dalam graf. Semakin tinggi rasio ini, semakin homofili graf tersebut.

Segmen Program 4.9 Evaluasi pengaruh parameter homofili koefisien pada dataset graph ShapeGGen

```

01:  # Fungsi untuk visualisasi dengan warna node berdasarkan
02:  labelnya
03:  def visualize_graph(dataset, title, ax):
04:      G = nx.Graph()
05:      edge_list = dataset.graph.edge_index.numpy().T
06:      G.add_edges_from(edge_list)
07:
08:      # Ambil label node
09:      labels = dataset.graph.y.tolist()
10:      # Buat warna berdasarkan label
11:      color_map = plt.get_cmap('coolwarm')
12:      unique_labels = sorted(set(labels))
13:      node_colors = [color_map(label / max(unique_labels)) for
14:                     label in labels]
15:
16:      # Posisi tetap agar hasil visualisasi konsisten
17:      pos = nx.spring_layout(G, seed=42)
18:      nx.draw(G, pos, node_color=node_colors,
19:             with_labels=False, node_size=250, edge_color='gray',
20:             alpha=0.6, ax=ax)
21:
22:      ax.set_title(title, fontsize=14)
23:
24:  def compute_homophily_ratio(dataset):
25:      """Menghitung homophily ratio dengan menampilkan jumlah
26:      edge antara label yang sama dan berbeda."""
27:
28:      G = nx.Graph()
29:      edge_list = dataset.graph.edge_index.numpy().T
30:      G.add_edges_from(edge_list)

```

Segmen Program 4.9 (Lanjutan)

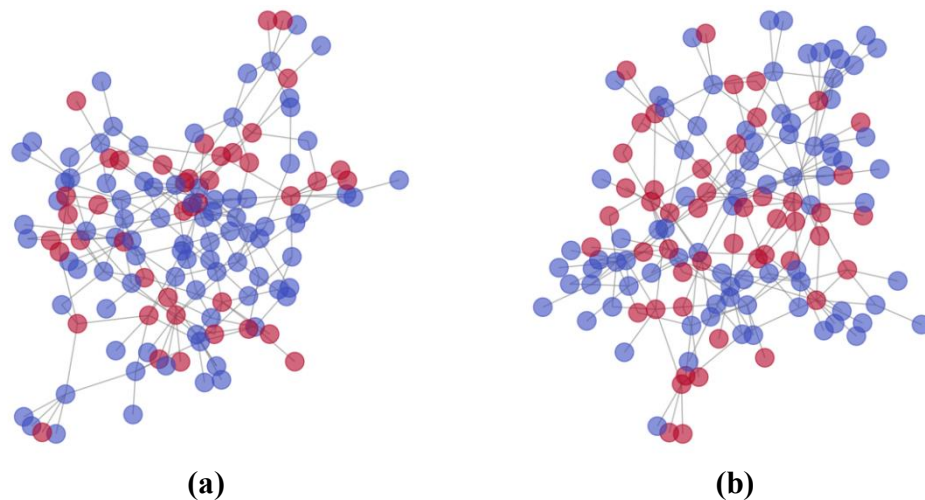
```

31:     labels = dataset.graph.y.numpy() # Label node
32:     same_label_edges = 0 # Jumlah edge dengan label yang
33:     sama
34:     diff_label_edges = 0 # Jumlah edge dengan label berbeda
35:     total_edges = G.number_of_edges()
36:     for u, v in G.edges():
37:         if labels[u] == labels[v]: # Jika kedua node
38:             memiliki label yang sama
39:                 same_label_edges += 1
40:         else:
41:             diff_label_edges += 1
42:
43:     homophily_ratio = same_label_edges / total_edges
44:
45:     print(f"Total Edges: {total_edges}")
46:     print(f"Edges with same label: {same_label_edges}")
47:     print(f"Edges with different label: {diff_label_edges}")
48:     print(f"Homophily Ratio: {homophily_ratio:.3f}")
49:
50:     return homophily_ratio
51:
52: torch.manual_seed(42)
53:
54: # Dataset Homofili ( $\eta = 1$ )
55: dataset_homo = ShapeGGen(S='house', Ns=1200, p=0.008, ns=11,
56:                          K=2, nf=11, ni=4, sf=0.8, cf=4,  $\phi$ =0.5,  $\eta$ =1,
57:                          L=3)
58:
59: # Dataset Heterofili ( $\eta = -1$ )
60: dataset_hetero = ShapeGGen(S='house', Ns=1200, p=0.008,
61:                            ns=11, K=2, nf=11, ni=4, sf=0.8, cf=4,
62:                             $\phi$ =0.5,  $\eta$ =-1, L=3)
63:
64: # Buat subplot untuk membandingkan kedua graf
65: fig, axes = plt.subplots(1, 2, figsize=(14, 7))
66:
67: # Visualisasi Graf
68: visualize_graph(dataset_homo, "", axes[0])
69: visualize_graph(dataset_hetero, "", axes[1])
70: plt.show()
71:
72: # Hitung rasio homofilik
73: print("\nHomophily Ratio untuk Graf Homofilik:")
74: homophily_homo = compute_homophily_ratio(dataset_homo)
75:
76: print("\nHomophily Ratio untuk Graf Heterofilik:")
77: homophily_hetero = compute_homophily_ratio(dataset_hetero)

```

Setelah itu pada baris 56-78, dua dataset dibuat menggunakan ShapeGGen, satu dengan $\eta = 1$ untuk graf homofili dan satu lagi dengan $\eta = -1$ untuk graf heterofili. Kedua graf kemudian divisualisasikan secara berdampingan menggunakan subplot, sehingga pengguna dapat langsung membandingkan pola

keterhubungan node dalam dataset homofili dan heterofili. Terakhir, rasio homofili untuk kedua dataset dihitung dan ditampilkan di terminal. Berikut adalah hasil visualisasi dari kedua dataset, bagian kiri adalah graf homofili dan bagian kanan adalah graf heterofili.



Gambar 4.8
Hasil Dataset Graf dengan Parameter Homofili Koefisien yang berbeda (a) Graf Homofili. (b) Graf Heterofili.

Pada Graf Homofili ($\eta = 1$), node lebih sering terhubung dengan label yang sama. Sedangkan Graf Heterofilk ($\eta = -1$), node lebih sering terhubung dengan label berbeda. Jika dilihat secara kasat mata, gambar di atas tidak terlihat jelas perbedaannya. Oleh karena itu, dapat dilakukan perhitungan rasio koneksi homofili yaitu jika hasil H mendekati 1, maka graf sangat homofili. Jika hasil H mendekati 0, graf sangat heterofili.

$$H = \frac{\text{Jumlah edge antara node dengan label sama}}{\text{total jumlah edge dalam graf}} \dots\dots\dots(4.1)$$

Dari Gambar 4.8, telah dihitung rasio homofilik dari 2 graf. diperoleh hasil sebagai berikut:

- Homophily Ratio untuk Graf Homofili:

Total Edges: 184

Edges with same label: 104

Edges with different label: 80

Homophily Ratio: 0.565

- Homophily Ratio untuk Graf Heterofili:

Total Edges: 188

Edges with same label: 90

Edges with different label: 98

Homophily Ratio: 0.479

Uji coba ini membuktikan bahwa Homophily Ratio dipengaruhi oleh koefisien homofili (η). Graf dengan $\eta = 1$ menunjukkan koneksi yang lebih kuat di antara node dengan label yang sama, sementara graf dengan $\eta = -1$ memiliki pola koneksi yang lebih acak, dengan kecenderungan tinggi untuk menghubungkan node dengan label berbeda.

4.7 Evaluasi Pengaruh Parameter Kelas dan Probabilitas ShapeGGen terhadap struktur Graph

Pengujian ini bertujuan untuk memahami bagaimana variasi jumlah kelas (K) dan probabilitas koneksi antar-subgraf (p) dalam ShapeGGen mempengaruhi struktur graf yang dihasilkan. Secara khusus, eksperimen ini akan membahas bagaimana perubahan jumlah kelas dapat mempengaruhi distribusi keterhubungan antar-node serta bagaimana probabilitas koneksi dapat menentukan tingkat kepadatan graf. Kode yang digunakan mirip seperti segmen program 4.9, namun terdapat penyesuaian dataset sebagai berikut.

Segmen Program 4.10 Dataset untuk evaluasi parameter K dan p pada ShapeGGen

```
01: # Dataset dengan jumlah kelas kecil (K = 2) dan besar (K = 5)
02: dataset_k2 = ShapeGGen(S='house', Ns=1200, p=0.01, ns=11,
03:                        K=2, nf=11, ni=4, sf=0.8, cf=4,  $\phi$ =0.5,  $\eta$ =1, L=3)
04:
05: dataset_k5 = ShapeGGen(S='house', Ns=1200, p=0.01, ns=11,
06:                        K=5, nf=11, ni=4, sf=0.8, cf=4,  $\phi$ =0.5,  $\eta$ =1, L=3)
07:
08:
09: # Dataset dengan probabilitas koneksi rendah (p = 0.005) dan
```

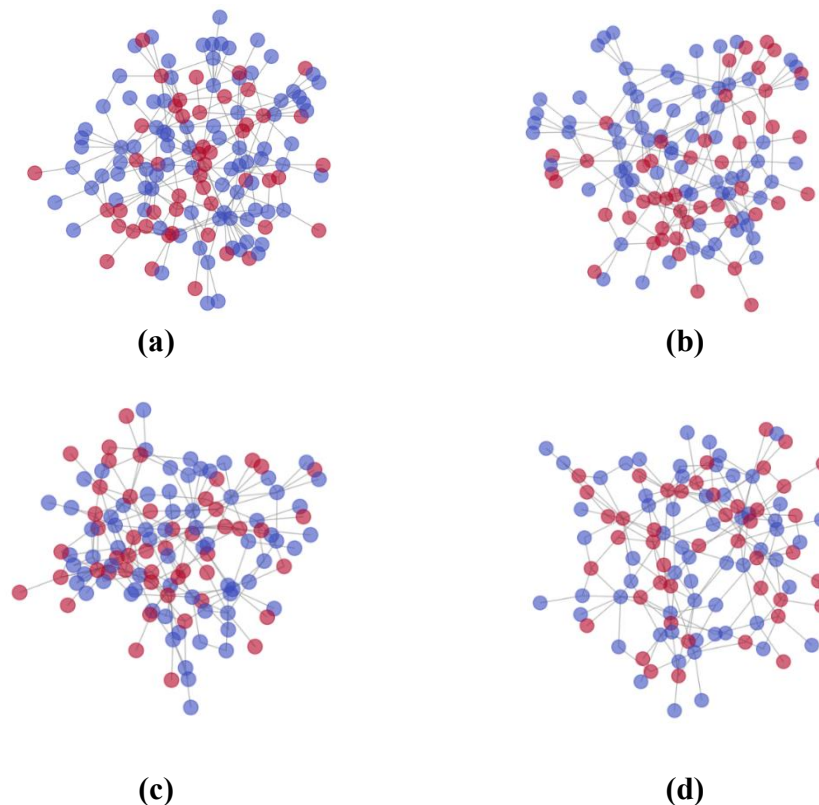
Segmen Program 4.10 (Lanjutan)

```

10: tinggi (p = 0.02)
11: dataset_p_low = ShapeGGen(S='house', Ns=1200, p=0.005, ns=11,
12:     K=2, nf=11, ni=4, sf=0.8, cf=4,  $\phi$ =0.5,  $\eta$ =1, L=3)
13:
14: dataset_p_high = ShapeGGen(S='house', Ns=1200, p=0.02, ns=11,
15:     K=2, nf=11, ni=4, sf=0.8, cf=4,  $\phi$ =0.5,  $\eta$ =1, L=3)

```

Pada parameter K (jumlah kelas dalam dataset), jika nilai K lebih tinggi, maka semakin banyak kelompok label yang berbeda dalam graf. Hal ini berpotensi mengurangi homofili, karena kemungkinan setiap node terhubung dengan node dari kelas lain meningkat. Sebaliknya, jika K lebih kecil, maka graf cenderung lebih homofilik.

**Gambar 4.9**

Hasil Dataset Graf dengan Parameter P dan K yang berbeda. (a) Graf Kelas K=2. (b) Graf Kelas K=5. (c) Graf Probabilitas $p=0.005$. (d) Graf Probabilitas $p=0.02$.

Pada parameter p (probabilitas koneksi antar-subgraf), jika p lebih tinggi, maka jumlah koneksi dalam graf akan meningkat, menyebabkan kepadatan graf menjadi lebih tinggi dan jalur komunikasi antar-node menjadi lebih pendek.

Namun, jika nilai p terlalu tinggi, struktur komunitas dalam graf dapat menjadi kurang jelas, sehingga sulit membedakan pola keterhubungan berdasarkan label kelas. Setelah generate graph berhasil, setiap grafnya dihitung homophily ratio dan densitynya dan menghasilkan hasil perhitungan seperti berikut:

- Analisis untuk Dataset dengan $K=2$:
 Total Edges: 192
 Edges with same label: 104
 Edges with different label: 88
 Homophily Ratio: 0.542
 Density: 0.021
- Analisis untuk Dataset dengan $K=5$:
 Total Edges: 175
 Edges with same label: 90
 Edges with different label: 85
 Homophily Ratio: 0.514
 Density: 0.025
- Analisis untuk Dataset dengan $p=0.005$:
 Total Edges: 175
 Edges with same label: 88
 Edges with different label: 87
 Homophily Ratio: 0.503
 Density: 0.025
- Analisis untuk Dataset dengan $p=0.02$:
 Total Edges: 162
 Edges with same label: 91
 Edges with different label: 71
 Homophily Ratio: 0.562
 Density: 0.029

Berdasarkan hasil eksperimen, terlihat bahwa peningkatan jumlah kelas (K) dan probabilitas koneksi antar-subgraf (p) mempengaruhi struktur graf secara

signifikan. Pada graf dengan $K=2$, total edge lebih banyak dibanding $K=5$ (192 vs. 175), tetapi Homophily Ratio lebih tinggi (0.542 vs. 0.514) karena node dengan label yang sama lebih sering terhubung. Density sedikit meningkat pada $K=5$ (0.025 vs. 0.021), menunjukkan bahwa meskipun koneksi antar-label lebih banyak, keterhubungan dalam subgraf menjadi lebih tersebar.

Sementara itu, pada pengaruh p terhadap struktur graf, peningkatan dari $p=0.005$ ke $p=0.02$ menyebabkan Homophily Ratio meningkat dari 0.503 menjadi 0.562, menunjukkan bahwa subgraf menjadi lebih terhubung secara internal. Density juga meningkat dari 0.025 menjadi 0.029, menandakan bahwa koneksi antar-node dalam subgraf bertambah, meskipun jumlah total edge justru sedikit berkurang dari 175 menjadi 162. Hal ini menunjukkan bahwa peningkatan p tidak selalu meningkatkan jumlah edge, tetapi mengubah pola koneksi dalam graf.

Secara keseluruhan, uji coba ini membuktikan bahwa semakin banyak jumlah kelas, semakin besar kemungkinan keterhubungan antar-label, menurunkan Homophily Ratio, sementara peningkatan p meningkatkan keterhubungan dalam subgraf, menaikkan Homophily Ratio dan Density.

Beberapa evaluasi di atas menunjukkan bahwa parameter-parameter dalam ShapeGGen dapat dikonfigurasi untuk menghasilkan berbagai jenis struktur graf, baik yang memiliki komunitas yang kuat (homofili) maupun yang lebih tersebar dengan interaksi antar-kelas yang lebih banyak (heterofili). Parameter seperti η , p , ns , cf , dan sf memiliki pengaruh langsung terhadap keterhubungan node, kepadatan edge, serta pola klasifikasi dalam graf. Oleh karena itu, dalam penggunaan ShapeGGen, pemilihan parameter yang tepat sangat penting untuk memastikan graf yang dihasilkan sesuai dengan tujuan penelitian, terutama dalam konteks pengujian model explainability dalam Graph Neural Networks (GNN).